

A1

Ссылка на репозиторий

Репозиторий - <https://github.com/loxxych/AaDS-SET9-A1>

В папке `StringSort` представлены реализации алгоритмов сортировки.

В папке `StringGenerator` представлена реализация класса-генератора массивов со строками.

В папке `StringSortTester` представлена реализация соответствующего класса-тестировщика.

В папке `Plotter` находится python-проект для отрисовки графиков.

В папке `Pltter/plots` представлены полученные графики.

В папке `TestResults` представлены результаты замеров.

Посылки на CodeForces

A1m - [375925429](#)

A1q - [375926955](#)

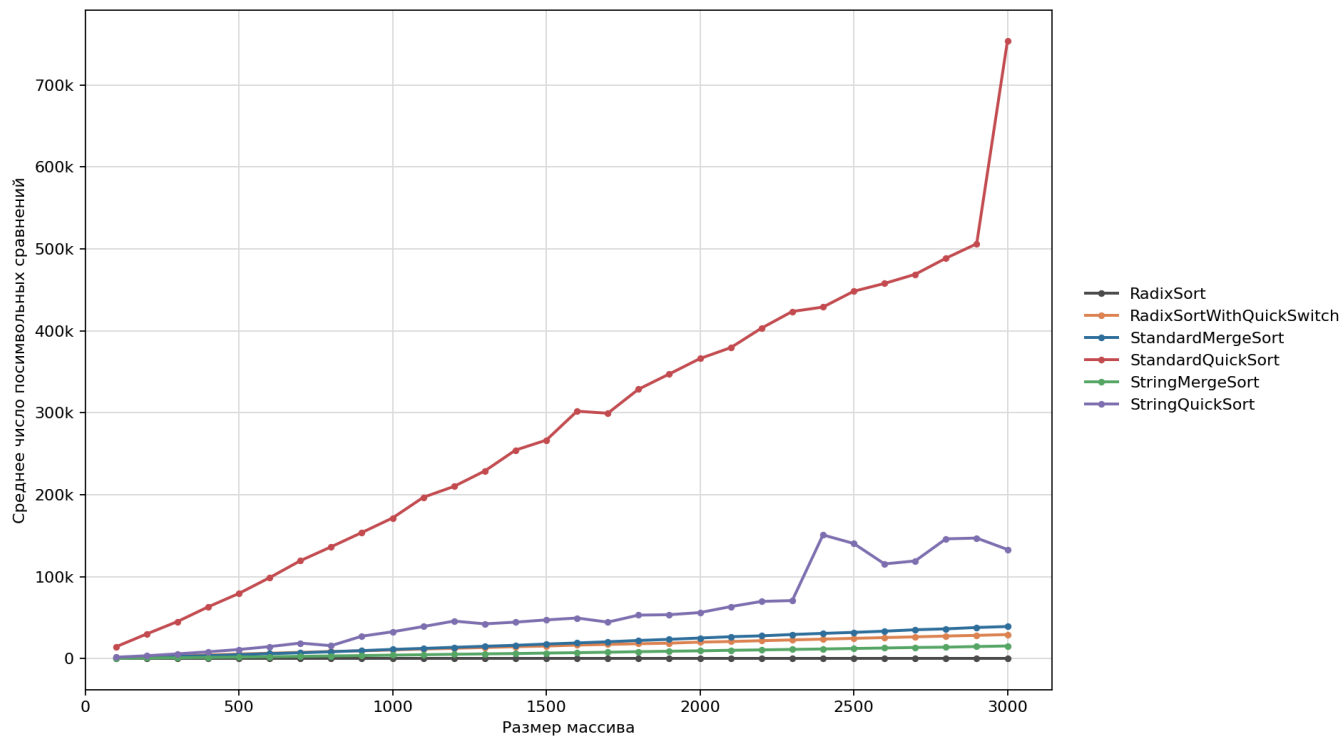
A1r - [375935359](#)

A1rq - [375935518](#)

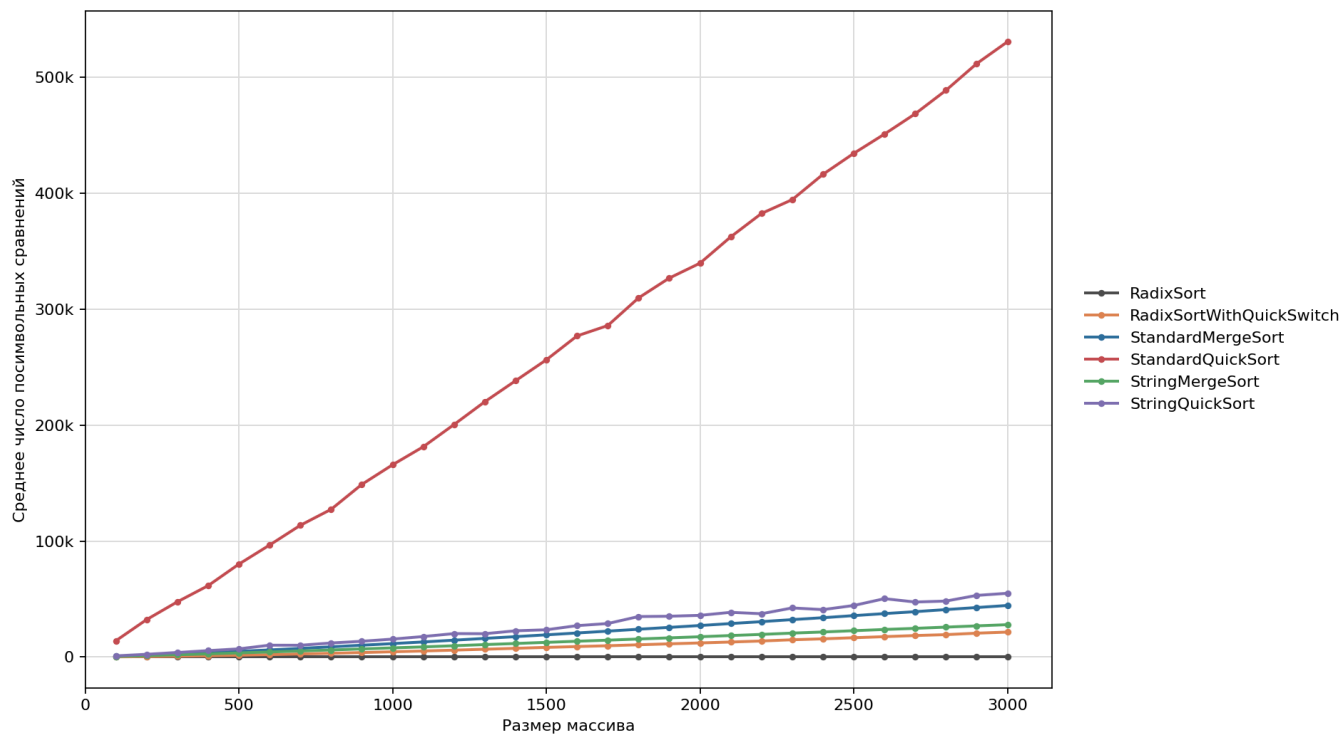
Анализ графиков

В ходе запуска программы были получены следующие результаты:

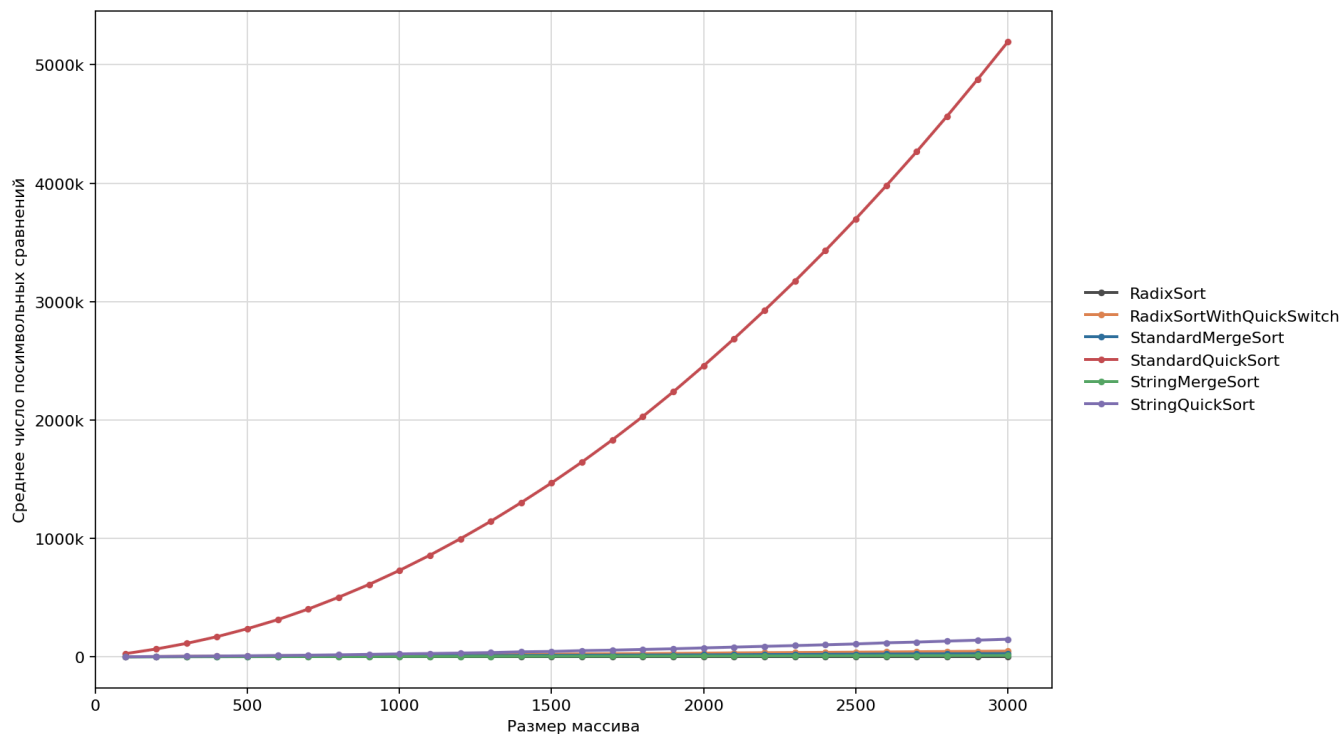
Среднее число посимвольных сравнений: почти отсортированные данные



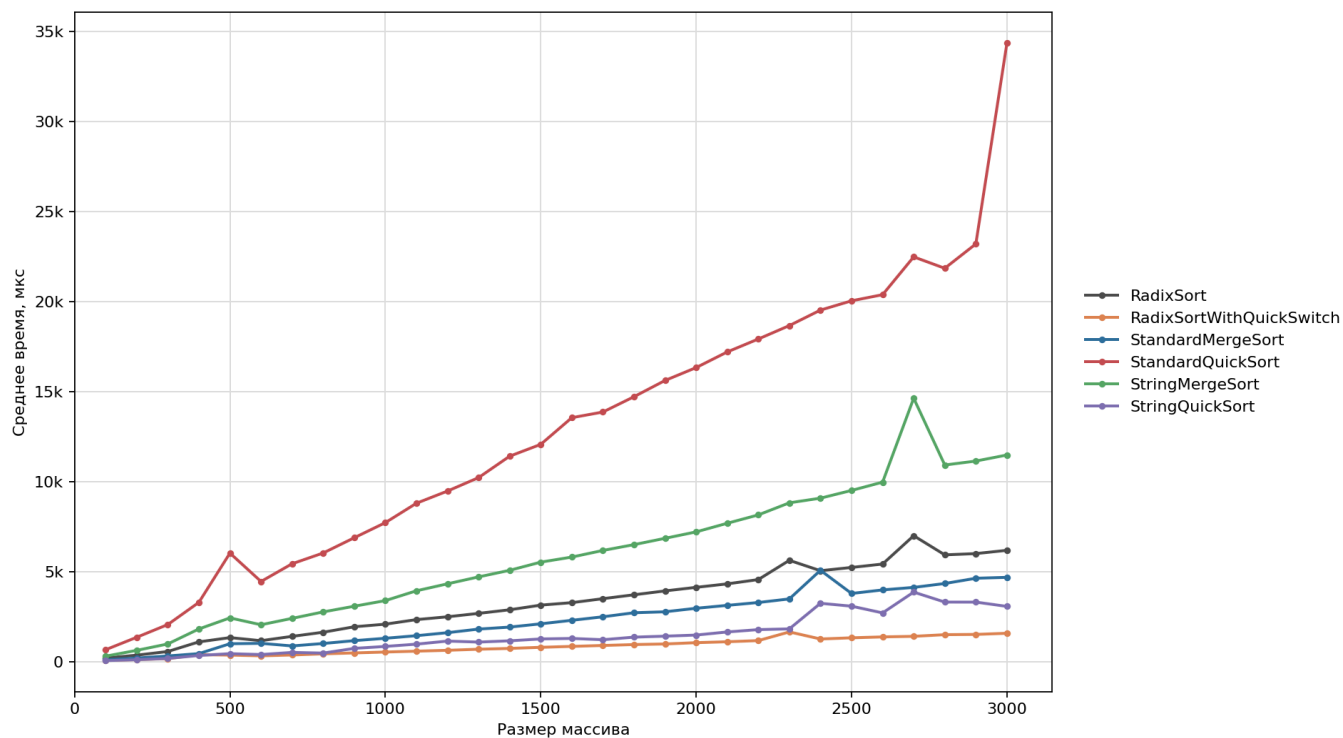
Среднее число посимвольных сравнений: случайные данные



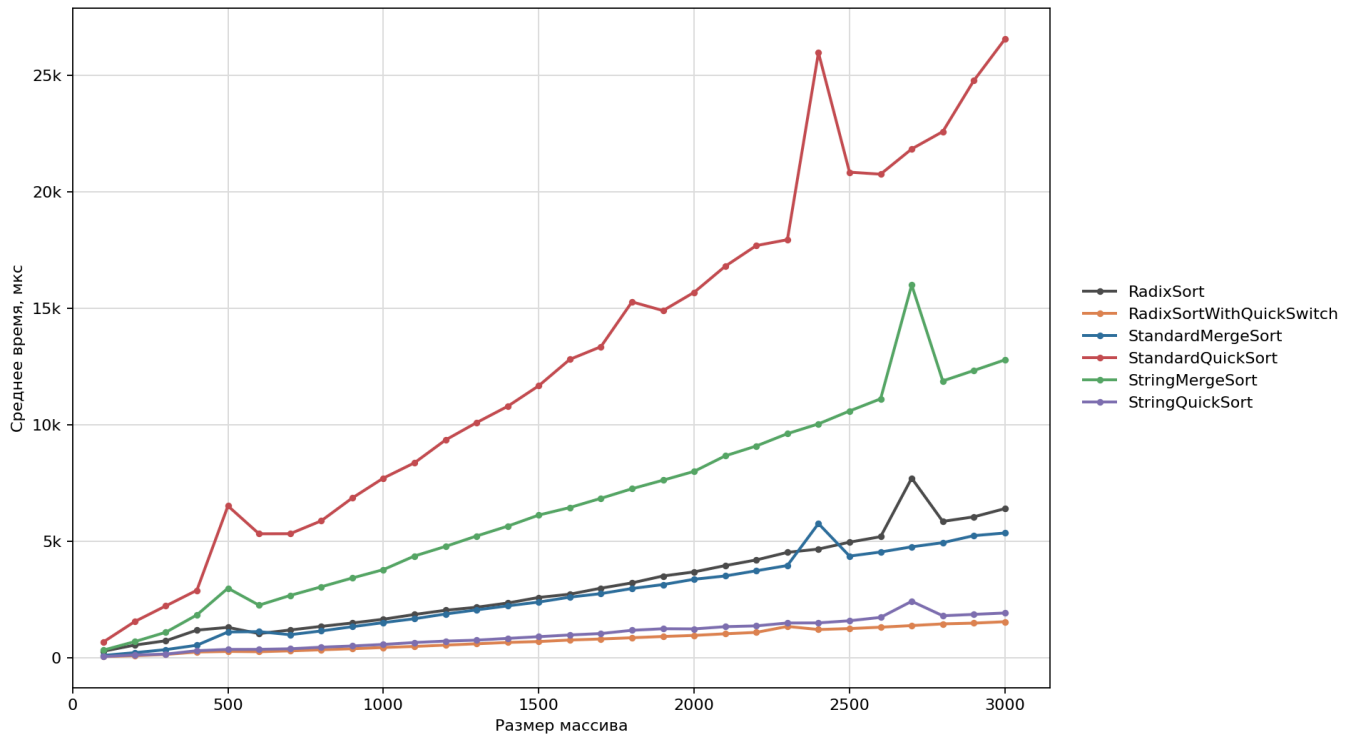
Среднее число посимвольных сравнений: обратно отсортированные данные



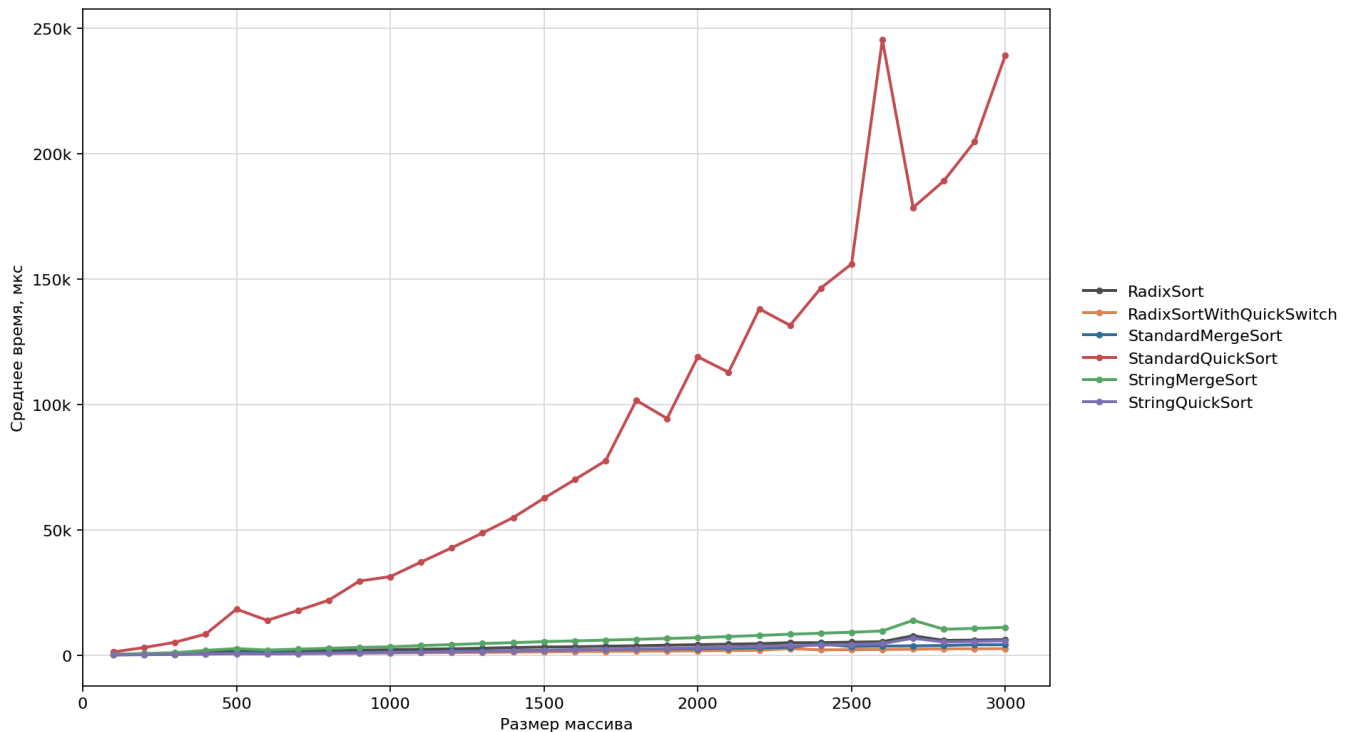
Среднее время, мкс: почти отсортированные данные



Среднее время, мкс: случайные данные



Среднее время, мкс: обратно отсортированные данные



По графикам видно, что стандартные алгоритмы сравнения строк заметно проигрывают специализированным строковым алгоритмам. Особенно сильно это проявляется у `StandardQuickSort` : на всех типах данных его время и число посимвольных сравнений растут быстрее остальных. На обратно отсортированных данных это выражено сильнее всего: к размеру массива 3000 время достигает примерно до 240 тыс. мкс, а число посимвольных сравнений превышает 5 млн. Это согласуется с тем, что quicksort с

выбором левого элемента как `pivot` на обратно отсортированных данных попадает в неблагоприятный случай (в реализации используется именно такой выбор `pivot'a`).

`StandardMergeSort` ведет себя значительно стабильнее, чем `StandardQuickSort`. Его рост близок к ожидаемому $O(n \log(n))$, без резких провалов на обратно отсортированных данных. Однако он все равно проигрывает лучшим строковым алгоритмам по времени, потому что каждое сравнение строк может требовать посимвольного прохода.

Лучший результат по времени почти везде показывает `RadixSortWithQuickSwitch`. Он особенно хорошо работает на случайных и почти отсортированных данных: при размере 3000 его время остается около 1.5 тыс. мкс, что намного меньше, чем у стандартных сортировок. Это объясняется тем, что MSD radix sort обрабатывает строки посимвольно по разрядам, а переключение на `QuickSort` на маленьких подмассивах уменьшает накладные расходы рекурсивного radix-разбиения.

Чистый `RadixSort` без переключения работает медленнее версии с переключением. На графиках времени он находится примерно на уровне `StandardMergeSort` или чуть хуже/лучше в зависимости от типа данных. Это объясняется тем, что без переключения алгоритм продолжает создавать массивы счетчиков и вспомогательные структуры даже для маленьких подмассивов, где это становится дороже, чем строковая быстрая сортировка.

По графикам посимвольных сравнений у `RadixSort` значение равно нулю, поскольку radix sort не сравнивает символы между строками, а только читает символы для распределения по корзинам. Поэтому по критерию именно посимвольных сравнений он оказывается лучшим, хотя по времени не всегда минимален из-за накладных расходов на подсчет и перераспределение.

`QuickSort` показывает хорошие результаты по времени и числу сравнений на случайных данных. Он значительно лучше `QuickSort`, потому что не сравнивает строки заново с начала при каждом сравнении, а сортирует по текущему символу и рекурсивно переходит к следующим символам только для равных префиксов. На почти отсортированных данных число сравнений у него заметно возрастает, особенно после размера около 2400, но по времени он все равно остается одним из лучших алгоритмов.

`MergeSort` использует информацию о LCP и действительно уменьшает число посимвольных сравнений по сравнению со стандартными алгоритмами, особенно на обратно отсортированных и почти отсортированных данных. Однако по времени он не всегда выигрывает: на графиках видно, что `MergeSort` часто медленнее `StandardMergeSort`. Это можно объяснить дополнительными накладными расходами на хранение LCP, создание блоков и работу с дополнительными структурами данных. То есть теоретическое уменьшение числа сравнений не полностью превращается в выигрыш по реальному времени.

Для случайных данных порядок по времени примерно такой: быстрее всего RadixSortWithQuickSwitch и QuickSort, затем StandardMergeSort и RadixSort, затем MergeSort, а самым медленным становится StandardQuickSort. Для обратно отсортированных данных разрыв особенно большой: StandardQuickSort резко деградирует, а специализированные алгоритмы сохраняют существенно меньший рост. Для почти отсортированных данных картина похожа на случайные данные, но у StandardQuickSort и QuickSort видны скачки, связанные с особенностями разбиения и выбранным pivot.

В целом результаты подтверждают теоретические ожидания: алгоритмы, учитывающие структуру строк и общие префиксы, уменьшают количество лишних посимвольных сравнений. Наиболее практичным по времени оказался MSD radix sort с переключением на QuickSort, а стандартный quicksort оказался самым нестабильным и чувствительным к типу входных данных.

Сравнение с теоретическими оценками адаптированных алгоритмов

QuickSort

Средняя сложность: $O(n \log n + L)$, где L - число просмотренных символов строк. В худшем случае при неудачных разбиениях возможна деградация до $O(n^2 + L)$. На графиках видно, что QuickSort выполняет намного меньше посимвольных сравнений, чем StandardQuickSort, особенно на случайных данных. Однако на почти отсортированных и обратно отсортированных данных рост становится заметнее из-за выбранного pivot.

MergeSort

Теоретическая сложность по структуре слияний остается $O(n \log n)$, но число посимвольных сравнений уменьшается за счет использования LCP. Если обозначить суммарную длину реально просмотренных префиксов как L , то строковая часть работы ближе к $O(n \log n + L)$, а не к полному повторному сравнению строк на каждом уровне. На графиках это видно по меньшему числу посимвольных сравнений относительно StandardMergeSort, хотя по времени выигрыш частично съедается накладными расходами на хранение и обработку LCP.

MSD RadixSort без переключения

Теоретическая сложность: $O(W + Rn)$ в практической форме, где W - суммарное число обработанных символов, R - размер алфавита. Т.к. в нашем случае алфавит фиксирован,

можно считать, что сложность $O(W)$. Алгоритм не выполняет сравнений строк между собой, поэтому на графиках число посимвольных сравнений равно нулю. По времени он растет плавно, но не всегда минимален из-за накладных расходов на массивы счетчиков и перераспределение.

MSD RadixSort с переключением на QuickSort

Теоретически основная часть остается близкой к $O(W)$ при фиксированном алфавите, но на маленьких подмассивах вместо дальнейшего radix-разбиения используется QuickSort. Это уменьшает константные накладные расходы. На графиках это подтверждается: версия с переключением почти всегда быстрее чистого RadixSort, хотя число посимвольных сравнений становится ненулевым из-за участка, где работает QuickSort.